

via GDB's call command. Then the *mmap* system call is used to map the just-opened *auxSymbols.o*. Once mapped, it is now part of the address space of the inferior process (*traceMe_rel*). Then debug symbols from *auxSymbols.o* are added to the symbol table using *add-symbol-file*. After this, GDB now knows what *auxST* is. The commands *call* and *add-symbol-file* were discussed in Part 1.

Also, the GDB variable '\$' represents the most recent displayed value. I have used this variable at two points:

1. To get the file descriptor returned by *open*, which is passed to *mmap* later on.
2. To get the address returned by *mmap*, at which *mmap* loaded *auxSymbols.o* in memory. This address is subsequently passed to *add-symbol-file*. Please refer to the documentation of *mmap* for the magic numbers used in the script. 2516 is the size of the *auxSymbols.o* object file.

The script executes the inferior process inside a 'while' loop, and emits a line starting with "in GDB:TRACER: " on every intercepted call to function *func*. All script output has "GDB: " to differentiate from the output of *traceMe_rel* (lines with the prefix "Entered ").

Now run *traceMe_rel* in GDB with the script *trace1.gdb* and redirect the output to a file named *out_trace1.txt*. The error printed is because the command at line 21 attempted to execute even after *traceMe_rel* had finished executing; ignore this for now!

```
[ranan@Chalotra Part2]$ gdb -x trace1.gdb -batch > out_trace1.txt
trace1.gdb:21: Error in sourced command file:
No registers.
```

Let's analyse the output of this command:

```
[ranan@Chalotra Part2]$ grep GDB out_trace1.txt
GDB:TRACER: 100, 000000000000100
GDB:TRACER: 25, 000000000000025
GDB:TRACER: 6, 000000000000006
GDB:TRACER: 1, 000000000000001
GDB:TRACER: 0, 000000000000000
```

Grepping "GDB:" gives us all the argument values traced by the script. To make sure that the script traced the correct values, given below is the output of *traceMe_rel*. Both outputs match, which means the script is working! You may like to look at *out_trace1.txt* for other output of the script.

```
[ranan@Chalotra Part2]$ grep Enter out_trace1.txt
Entered [1 ] with st->val=100 , st-
>arrayArg=000000000000100
Entered [2 ] with st->val=25 , st-
>arrayArg=000000000000025
```

```
Entered [3 ] with st->val=6 , st-
>arrayArg=000000000000006
Entered [4 ] with st->val=1 , st-
>arrayArg=000000000000001
Entered [5 ] with st->val=0 , st-
>arrayArg=000000000000000
```



Note: Any attempt to use anything other than the debug symbols from *auxSymbols.o* will fail, because symbols in *auxSymbols.o* have not been relocated/resolved into the inferior address space yet.

'trace2.gdb'

This script uses the *LD_PRELOAD* variable to load *libauxSym.so* into the address space of *traceMe_rel*. The rest is similar to *trace1.gdb*. Here is *trace2.gdb*:

```
set environment LD_PRELOAD ./libauxSym.so
file ./traceMe_rel
br main
run 100
bt
ptype auxST
ptype func
br func
c
while 1
set variable $one = ((int *)($ebp +0x8))

printf "\n GDB:TRACER: %d, %s\n", ((auxST *) ($one))->auxVal,
((auxST *) ($one))->auxArray
c
end
```

Run *traceMe_rel* using *trace2.gdb*, redirecting output to *out_trace2.txt*, with *gdb -x trace2.gdb -batch > out_trace2.txt*. Analyse the output; it is the same as *out_trace1.txt*:

```
[ranan@Chalotra Part2]$ grep GDB out_trace2.txt
GDB:TRACER: 100, 000000000000100
GDB:TRACER: 33, 000000000000033
GDB:TRACER: 11, 000000000000011
GDB:TRACER: 3, 000000000000003
GDB:TRACER: 0, 000000000000000

[ranan@Chalotra Part2]$ grep Enter out_trace2.txt
Entered [1 ] with st->val=100 , st-
>arrayArg=000000000000100
Entered [2 ] with st->val=33 , st-
>arrayArg=000000000000033
Entered [3 ] with st->val=11 , st-
>arrayArg=000000000000011
Entered [4 ] with st->val=3 , st-
```